

Tara Travel - One-Page App Summary

Evidence-based summary generated from repo files inspected in this workspace.

What It Is

Tara Travel is a Flutter app for coordinating shared trips in one place, with screens for planning, budgets, members, packing, navigation, chat, and notifications.

Repo evidence shows Supabase-backed auth/data plus a local Sembast cache for selected trip, itinerary, expense, and profile data.

Who It's For

Primary persona: a small-group trip organizer traveling with friends or family who needs to coordinate plans, people, and shared costs.

Formal persona document: Not found in repo.

What It Does

- Supports onboarding plus Magic Link, email/password, and Google OAuth sign-in.
- Creates trips through a 4-step flow for details, transport, budget, and confirmation.
- Tracks itineraries, stop status, and trip activity logs.
- Manages shared budgets, expenses, contributions, split modes, and settlements.
- Coordinates members with invite codes, trip roles, and contact actions.
- Keeps shared packing lists with suggestions, progress, and reminder actions.
- Includes screens for notifications, group chat, and live/group navigation UI.

Key repo evidence: lib/main.dart, Riverpod providers/repositories, Supabase migrations, Supabase Edge Functions, and .env.example.

How It Works

- Flutter MaterialApp routes into feature screens; Riverpod providers expose app state and repositories.
 - Repositories call Supabase for auth and trip data, then cache selected records locally in Sembast for offline fallback and optimistic UI.
 - Supabase migrations define users, trips, trip_members, itinerary_stops, expenses, settlements, packing_items, contributions, activity_log, notifications, destinations, member_locations, and trip_messages with row-level security.
 - Supabase Edge Functions send Expo push notifications directly and when expense status changes.
- Concrete data-flow diagram or architecture doc: Not found in repo.

How To Run

- Install Flutter with a Dart SDK compatible with $\geq 3.2.0$ and $< 4.0.0$.
 - Copy .env.example to .env and set EXPO_PUBLIC_SUPABASE_URL plus EXPO_PUBLIC_SUPABASE_ANON_KEY.
 - Apply Supabase SQL migrations 001_initial_schema.sql and 002_real_data_support.sql. Optional dev data: 003_dev_seed.sql.
 - Run flutter pub get, then flutter run from the repo root.
- Notification/Edge Function deployment steps beyond the source files: Not found in repo.